ELA: Executable Logic for Arithmetic Descent Certificates

A Certificate-Oriented Computational Architecture for Collatz-Type Descent Tracing

Version 1.3.1

Onur Seçilmiş
ORCID: 0009-0004-2553-2262

Abstract

ELA, short for *Executable Logic for Arithmetic Descent*, is a certificate-oriented computational architecture for tracing arithmetic descent in iterative numerical processes. The system does not claim to prove the Collatz conjecture or any global convergence theorem. Its narrower contribution is engineering-oriented: for a given input, ELA produces an exact integer trace, diagnostic telemetry, growth-corridor markers, stress readings, and a machine-checkable local descent certificate when the trajectory first falls below its starting value.

The representative test map is

$$T_{a,b}(n) = \frac{an+b}{2^{v_2(an+b)}},$$

where a, b, n are positive odd integers. For $n_j = T_{a,b}^j(n_0)$, define the local balance

$$b_j = v_2(an_j + b) - \log_2\left(a + \frac{b}{n_j}\right).$$

Then

$$B_{a,b}(n_0, k) = \sum_{j=0}^{k-1} b_j = \log_2\left(\frac{n_0}{T_{a,b}^k(n_0)}\right).$$

Thus $B_{a,b}(n_0, k) > 0$ is equivalent to the exact local descent condition $T_{a,b}^k(n_0) < n_0$. ELA separates proof-level integer checks from floating-point telemetry: plots and balances are diagnostic, while the certificate is the exact replay of integer transitions and the exact integer comparison.

1 Purpose, Boundary, and Prior Work

The purpose of ELA is not to announce a universal proof. The purpose is to turn a difficult iterative numerical process into a reproducible engineering object:

$$\text{input} \to \text{exact trace} \to \text{diagnostics} \to \text{certificate} \to \text{strict verification}.$$

The paper makes only the following claims:

1. Exact integer transitions can be recorded for a Collatz-type odd-affine divisive map.
2. Local growth load and 2-adic reduction credit can be separated as diagnostic telemetry.
3. A first-descent event can be certified by exact integer replay and comparison.
4. Sustained weak-reduction growth in the $a = 3, b = 1$ test map has a strict growth-corridor condition.
5. A short independent verifier can reject altered or fake certificates.

The paper does not claim novelty for the Collatz map, 2-adic valuation, checkpointing, tracing, trace compression, or distributed snapshot ideas. Those are prior concepts. The contribution here is the certificate-oriented integration: trace schema, balance identity, growth-corridor check, stress profile, descent certificate, and strict verifier in one executable architecture.

2 Trace Model

For a process

$$x_{j+1} = F(x_j),$$

an ELA trace record is

$$\text{TraceStep}_j = (j, x_j, x_{j+1}, m_j, B_j, \text{cert}_j, \text{flags}_j).$$

For the odd-affine divisive test map this becomes

$$\text{TraceStep}_j = (j, n_j, n_{j+1}, v_j, g_j, b_j, B_{j+1}, E_{j+1}, \text{cert}_j, \text{flags}_j).$$

Here v_j is the 2-adic reduction credit, g_j is the growth load, b_j is the local balance, B_{j+1} is the cumulative balance after the transition, E_{j+1} is stress relative to the starting value, and cert_j marks exact descent.

The design rule is:

floating-point telemetry is diagnostic; exact integer replay is the certificate.

3 Representative Map

Let a and b be positive odd integers and let n be a positive odd integer. Define

$$T_{a,b}(n) = \frac{an+b}{2^{v_2(an+b)}}.$$

Since a, b, n are odd, $an + b$ is even and $v_2(an + b) \geq 1$. After division by all powers of two, the output is again odd.

The one-step logarithmic ratio is

$$\log_2\left(\frac{T_{a,b}(n)}{n}\right) = \log_2\left(a + \frac{b}{n}\right) - v_2(an+b).$$

Equivalently,

$$v_2(an+b) - \log_2\left(a + \frac{b}{n}\right) = \log_2\left(\frac{n}{T_{a,b}(n)}\right).$$

This is an exact identity from the definition of the map.

4 Critical-Balance Certificate

Let

$$n_j = T_{a,b}^j(n_0).$$

Define

$$v_j = v_2(an_j + b), \qquad g_j = \log_2\left(a + \frac{b}{n_j}\right), \qquad b_j = v_j - g_j.$$

Then

$$b_j = \log_2\left(\frac{n_j}{n_{j+1}}\right).$$

The cumulative balance over k odd transitions is

$$B_{a,b}(n_0, k) = \sum_{j=0}^{k-1} b_j.$$

The sum telescopes:

$$B_{a,b}(n_0,k)=\sum_{j=0}^{k-1}\log_2\left(\frac{n_j}{n_{j+1}}\right)=\log_2\left(\frac{n_0}{n_k}\right)=\log_2\left(\frac{n_0}{T_{a,b}^k(n_0)}\right).$$

Therefore,

$$B_{a,b}(n_0,k)>0 \quad\Longleftrightarrow\quad T_{a,b}^k(n_0)<n_0.$$

This is a local descent certificate. It does not imply that such a k exists for every n_0.

5 Growth-Corridor Constraint

For the representative $a=3,b=1$ test map,

$$T(n)=\frac{3n+1}{2^{v_2(3n+1)}}.$$

A one-step increase $T(n)>n$ is possible if and only if

$$v_2(3n+1)=1,$$

which for odd n is equivalent to

$$n\equiv 3 \pmod 4.$$

Thus one-step growth is not free; it requires a precise residue class.

More strongly, at least r consecutive weak-reduction steps occur if and only if

$$n_0\equiv -1 \pmod{2^{r+1}},$$

or equivalently

$$2^{r+1}\mid (n_0+1).$$

If

$$n_0=2^{r+1}q-1,$$

then after r weak-reduction growth steps,

$$n_r=2\cdot 3^r q-1.$$

For exactly r consecutive weak-reduction steps, one additionally requires

$$n_0\not\equiv -1 \pmod{2^{r+2}}.$$

This is the growth-corridor constraint. It shows that sustained local growth requires increasingly restrictive 2-adic structure in the starting value. It is a structural condition, not a convergence proof.

6 Stress Trace

ELA records a stress trace after k transitions:

$$E_k=\log_2\left(\frac{n_k}{n_0}\right).$$

Using the balance identity,

$$E_k=-B_{a,b}(n_0,k).$$

Positive stress means the current state is above the starting value; negative stress means it is below. The first exact local certificate is still the integer condition

$$n_k<n_0.$$

Thus the stress trace is a diagnostic re-reading of the same exact identity, not a separate proof mechanism.

7 Minimal Embedded Software Core

The full validation script is included in the release bundle. The embedded core below is intentionally short, but stricter than a demo verifier: it rejects empty certified traces, step-length mismatches, altered rows, fake final values, invalid statuses, and altered certificate flags.

Listing 1: ELA minimal core with strict certificate verifier.

```
#!/usr/bin/env python3
"""ELA minimal executable core v1.3.1.

Executable Logic for Arithmetic Descent Certificates.
This file emits and strictly verifies local descent certificates
for the accelerated odd 3n+1 map.

Boundary: this program does not prove the Collatz conjecture.
It verifies a finite local descent certificate for each supplied input.
"""

VALID_STATUSES = {"certified", "terminal", "open_in_limit"}


def v2(m: int) -> int:
    """Return the exponent of 2 dividing positive integer m."""
    if not isinstance(m, int) or m <= 0:
        raise ValueError("positive integer expected")
    return (m & -m).bit_length() - 1


def step(n: int, a: int = 3, b: int = 1):
    """One accelerated odd-affine divisive step."""
    if not isinstance(n, int) or n <= 0 or n % 2 == 0:
        raise ValueError("n must be a positive odd integer")
    if a <= 0 or b <= 0 or a % 2 == 0 or b % 2 == 0:
        raise ValueError("a and b must be positive odd integers")
    m = a * n + b
    s = v2(m)
    return m >> s, s


def descent_certificate(n0: int, limit: int = 10000):
    """Emit a finite local descent certificate when n falls below n0."""
    if not isinstance(n0, int) or n0 <= 0 or n0 % 2 == 0:
        raise ValueError("n0 must be a positive odd integer")
    if not isinstance(limit, int) or limit <= 0:
        raise ValueError("limit must be a positive integer")

    n = n0
    trace = []
    for j in range(limit):
        nxt, s = step(n)
        row = {"j": j, "n": n, "next": nxt, "v": s, "cert": nxt < n0}
        trace.append(row)
        if nxt < n0:
            return {"n0": n0, "status": "certified", "step": j + 1,
                    "value": nxt, "trace": trace}
        if nxt == 1:
            return {"n0": n0, "status": "terminal", "step": j + 1,
                    "value": 1, "trace": trace}
        n = nxt

    return {"n0": n0, "status": "open_in_limit", "step": limit,
            "value": n, "trace": trace}


def verify_certificate(cert: dict) -> bool:
    """Strictly replay and verify an emitted certificate.

    The verifier rejects empty certified traces, step-length mismatches,
    altered rows, fake final values, and invalid statuses.
    """
    if not isinstance(cert, dict):
```

```
        raise AssertionError("certificate must be a dictionary")

    n0 = cert.get("n0")
    status = cert.get("status")
    step_count = cert.get("step")
    value = cert.get("value")
    trace = cert.get("trace")

    if not isinstance(n0, int) or n0 <= 0 or n0 % 2 == 0:
        raise AssertionError("invalid n0")
    if status not in VALID_STATUSES:
        raise AssertionError("invalid status")
    if not isinstance(trace, list):
        raise AssertionError("trace must be a list")
    if step_count != len(trace):
        raise AssertionError("step must equal trace length")

    n = n0
    last_next = None
    for expected_j, row in enumerate(trace):
        nxt, s = step(n)
        assert row.get("j") == expected_j
        assert row.get("n") == n
        assert row.get("next") == nxt
        assert row.get("v") == s
        assert row.get("cert") == (nxt < n0)
        last_next = nxt
        n = nxt

    if status == "certified":
        if not trace:
            raise AssertionError("certified certificate cannot have empty trace")
        assert value == last_next
        assert last_next < n0
        assert trace[-1].get("cert") is True

    elif status == "terminal":
        if not trace:
            raise AssertionError("terminal certificate cannot have empty trace")
        assert value == last_next
        assert last_next == 1

    elif status == "open_in_limit":
        assert value == n
        if trace:
            assert trace[-1].get("cert") is False
            assert last_next != 1

    return True


if __name__ == "__main__":
    for n0 in [7, 27, 97, 871, 6171, 77031]:
        cert = descent_certificate(n0)
        verify_certificate(cert)
        print(n0, cert["status"], cert.get("step"), cert.get("value"))
```

8 Validation Audit

The accompanying validation script checks the following statements by exact integer transitions and floating-point audits:

1. local identity: $b_j = \log_2(n_j/n_{j+1})$;
2. telescoping identity: $B_{a,b}(n_0, k) = \log_2(n_0/n_k)$;
3. stress relation: $E_k = -B_{a,b}(n_0, k)$;
4. descent certificate flag: $\mathrm{cert}_j \iff n_{j+1} < n_0$;

5. growth-corridor sufficiency and bounded equivalence;

6. strict verifier rejection of adversarial/tampered certificates.

The v1.3.1 audit passed all checks. The maximum local identity error was 4.441×10^{-16}; the maximum telescoping identity error was 7.105×10^{-15}; the maximum stress relation error was 7.105×10^{-15}. These are floating-point telemetry errors only. Certificate validity is exact integer replay and comparison.

The growth-corridor audit checked 896 sufficiency examples, 229376 bounded equivalence checks over odd inputs below 32768, and 8192 exact weak-run length checks. The strict verifier rejected the following tamper cases: empty certified trace, wrong final value, step length mismatch, altered transition row, altered certificate flag, and invalid status.

n_0	classical steps	classical peak	odd trans.	first cert.	cert. value	odd peak	peak stress
7	16	52	5	4	5	17	1.280108
27	111	9232	41	37	23	3077	6.832421
97	118	9232	43	1	73	3077	4.987396
871	178	190996	65	22	797	63665	6.191684
6171	261	975400	96	36	3215	325133	5.719382
77031	350	21933016	129	56	32567	7311005	6.568487

Table 1: Representative benchmark outputs emitted by the v1.3.1 validation script.

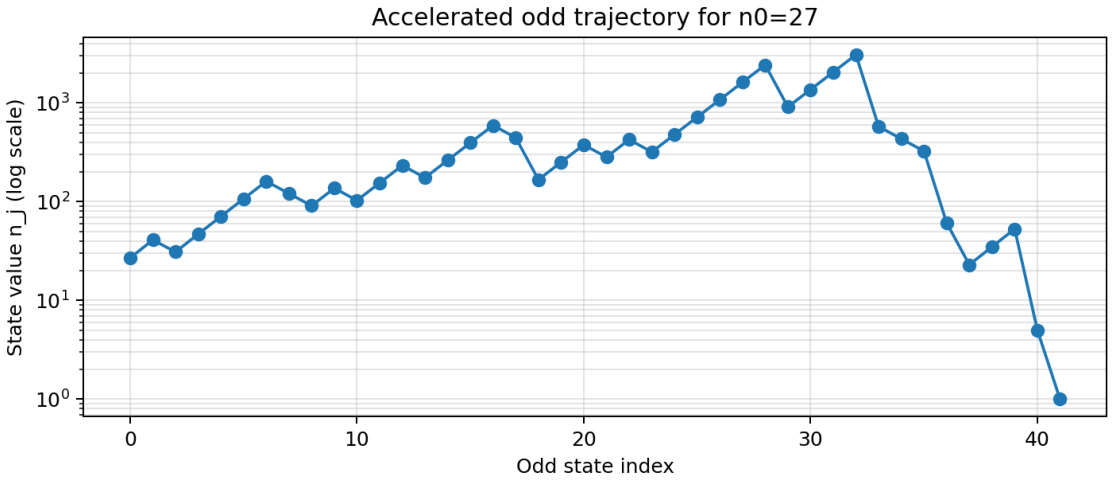

Figure 1: Accelerated odd trajectory for $n_0 = 27$. The plot is diagnostic, not proof-level evidence.

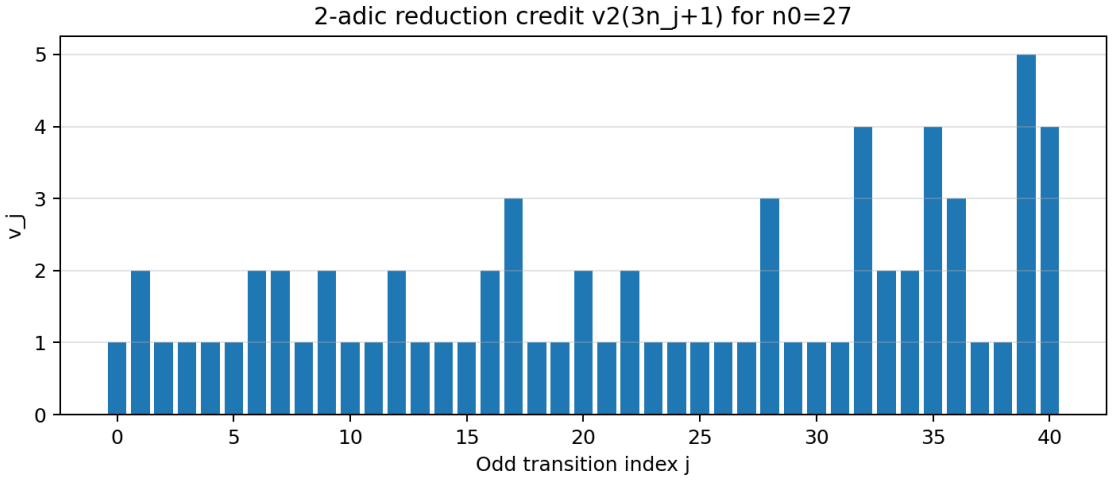

Figure 2: Reduction-credit sequence $v_j = v_2(3n_j + 1)$ for $n_0 = 27$. Height-1 bars are weak-reduction steps; larger bars are stronger reduction events.

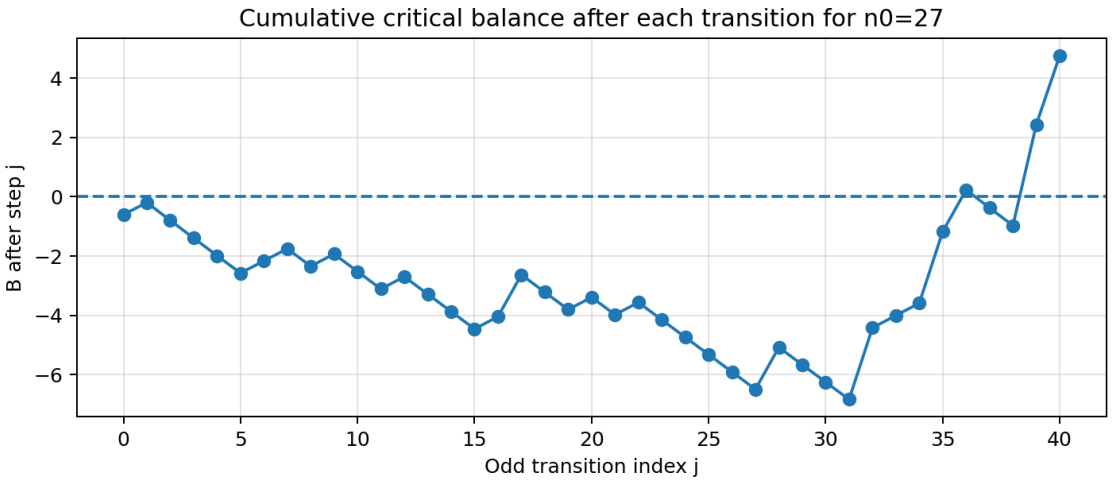

Figure 3: Cumulative critical balance after each transition for $n_0 = 27$. The certificate remains the exact integer condition $n_k < n_0$.

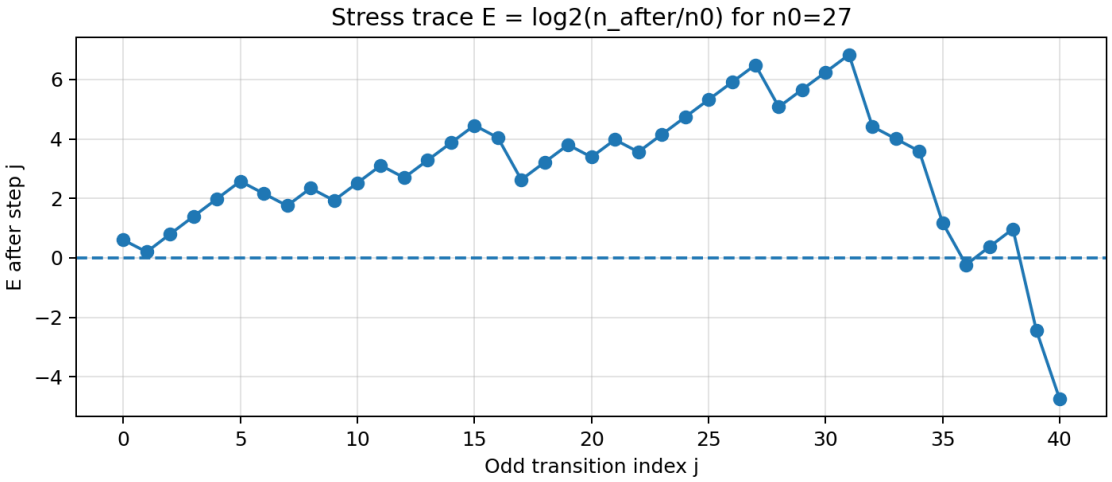

Figure 4: Stress trace $E_k = \log_2(n_k/n_0)$ for $n_0 = 27$. Positive regions mark excursions above the start. Since $E_k = -B_{a,b}(n_0, k)$, this is a diagnostic re-reading of the same identity.

9 Limitations

The system is intentionally bounded by the following limitations:

1. ELA does not prove that every starting value has a descent certificate.
2. The existence of a local certificate for one input does not imply a global convergence theorem.
3. Floating-point balance and plotted stress are telemetry, not proof-level evidence.
4. The growth-corridor condition explains sustained weak-reduction growth; it does not rule out every possible infinite behavior.
5. The embedded code is a minimal core; the full validation script is the reproducibility artifact.

These limitations are part of the claim boundary. The engineering result is a certificate-producing architecture, not an announced solution to an open problem.

10 Reproducibility Bundle

The reproducibility bundle contains:

- this PDF;
- the LaTeX source;
- the minimal executable core;
- the full validation script;
- the validation report;
- the benchmark CSV;
- generated figures;
- README and metadata files.

11 Conclusion

ELA reframes arithmetic descent as an executable certificate problem. For a given input, the system records exact transitions, separates growth load from reduction credit, produces cumulative balance telemetry, marks growth-corridor and stress events, and emits a strictly verifiable descent certificate when the trajectory first falls below its starting value.

The result is not a global proof. Its contribution is narrower and more engineering-focused: it turns a Collatz-type numerical trajectory into a structured, auditable, certificate-oriented execution trace with an independent verifier that rejects tampered certificates.

References

[1] J. C. Lagarias, "The $3x + 1$ problem and its generalizations," *The American Mathematical Monthly*, vol. 92, no. 1, pp. 3–23, 1985.

[2] J. C. Lagarias, editor, *The Ultimate Challenge: The* $3x + 1$ *Problem*. American Mathematical Society, 2010.

[3] T. Tao, "Almost all orbits of the Collatz map attain almost bounded values," *Forum of Mathematics, Pi*, vol. 10, e12, 2022.

[4] K. M. Chandy and L. Lamport, "Distributed snapshots: Determining global states of distributed systems," *ACM Transactions on Computer Systems*, vol. 3, no. 1, pp. 63–75, 1985.

[5] Z. Chen, J. Pu, and Z. Zheng, "Tracezip: Efficient distributed tracing via trace compression," arXiv:2502.06318, 2025.

[6] G. Bruni and H. T. Johansson, "DPTC – an FPGA-based trace compression," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 1, pp. 189–197, 2020.

[7] X. Zeng and S. Zhang, "CStream: Parallel data stream compression on multicore edge devices," *IEEE Transactions on Knowledge and Data Engineering*, 2024.